

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 11 月 15 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: CPU 设计之中断

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 张立冬、马致远

实验地点: 东 4-509 实验日期: 2024 年 11 月 27 日

一、实验目的和要求

1. 深入理解 CPU 结构
3. 学习如何提高 CPU 使用效率
3. 学习 CPU 中断工作原理
4. 设计中断测试程序
5. 熟悉 RISC-V 中断的原理, 了解引起 CPU 中断产生的原因及其处理方法, 扩展包含中断的 CPU

二、实验内容和原理

内容:

任务一: 扩展实验 CPU 中断功能

- 修改设计数据通路和控制器 (增加中断控制)
- 拓展 CPU 中断功能

任务二: 设计 CPU 中断测试方案并完成测试

原理：

2.1 中断基本概念

中断是指程序执行过程中，当发生某个事件时，中止 CPU 上 现程序的运行，引出处理该事件的程序执行的过程，此过程都需要打断处理器正常的工作。

- 中断源：引起中断的事件称为中断源；
- 中断请求：中断源向 CPU 提出处理的请求；
- 断点：发生中断时被打断程序的暂停点；
- 中断响应：CPU 暂停现程序而转为响应中断请求的过程；
- 中断处理程序：处理中断源的程序；
- 中断处理：CPU 执行有关的中断处理程序；
- 中断返回：返回断点的过程；

当 CPU 收到中断或者异常的信号时，它会暂停执行当前的程序或任务，通过一定的机制跳转到负责处理这个信号的相关处理程序中，在完成对这个信号的处理后再跳回到刚才被打断的程序或任务中。

- 保护 CPU 现场 --> • 处理发生的中断事件 --> • 恢复正常操作

2.2 RISC-V 中断处理寄存器

RISC-V 架构主要分为 3 个工作模式，不同模式均可产生中断：

- 机器模式(Machine Mode) —— 本实验目标
- 用户模式(User Mode)
- 监督模式(Supervisor Mode)

中断处理时涉及多个新寄存器，在此做简单介绍：

(1) 异常入口基址寄存器 - mtvec:

处理器进入异常后，跳入的 PC 地址由 mtvec 指定，本实验采取固定向量。一般，MODE=1 时进行中断的跳转，且跳转地址为 $BASE + 4 * CAUSE$ (CAUSE 为中断对应的编号)。

31	21	0
BASE		MODE

向量地址	ARM异常名称	ARM系统工作模式	本实验定义
0x0000000	复位	超级用户Svc	复位
0x0000004	未定义指令终止	未定义指令终止Und	非法指令异常
0x0000008	软中断（SWI）	超级用户Svc	ECALL
0x000000c	Prefetch abort	指令预取终止Abt	Int外部中断（硬件）
0x0000010	Data abort	数据访问终止Abt	Reserved自定义
0x0000014	Reserved	Reserved	Reserved自定义
0x0000018	IRQ	外部中断模式IRQ	Reserved自定义
0x000001C	FIQ	快速中断模式FIQ	Reserved自定义

【ARM 中断处理跳转地址向量表】

（2）异常原因寄存器 – mcause：

处理器进入异常后，异常的引发原因由 mcause 寄存器指定。

31			30			0		
interrupt			Exception code					

INT	EC	Description
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned

INT	EC	Description
0	7	Store/AMO access fault
0	10..8	Not supported
0	11	Ecall from M-mode
0	>=12	Reserved
1	2..0	Reserved
1	3	Machine Software Interrupt
1	6..4	Reserved
1	7	Machine Timer Interrupt
1	10..8	Reserved
1	11	Machine External Interrupt
1	>=12	Reserved

（3）*异常 PC 寄存器 – mepc：

处理器进入异常后，异常的返回地址由 mepc 寄存器保存。

在进入异常时，硬件将自动更新 mepc 寄存器的值，为当前遇到异常的指令 PC 的下一条指令。Mepc 寄存器作为异常的返回地址，在异常结束后，能够使用它保存的 PC 值回到之前被停止执行的程序点。

（4）异常值寄存器 – mtval：

处理器进入异常后，异常的存储器访问地址或指令编码由 mtval 寄存器保存。

（5）异常状态寄存器 – mstatus：

处理器进入异常后，异常的各种状态由 mstatus 寄存器指示。

2.3 RISC-V 中断处理相关指令

(1) 异常返回 – mret : $PC \leq mepc$

0011000	00010	00000	000	00000	1110011	R mret
---------	-------	-------	-----	-------	---------	--------

当异常程序处理完成后，最终要从异常服务程序中退出，并返回主程序。在机器模式下退出异常时候，软件须使用 MRET，规定：

- 停止执行当前程序流，转而从 csr 寄存器 mepc 定义的 pc 地址开始执行 •

执行 mret 指令不仅会让处理器跳转到上述的 pc 地址开始执行，还会让硬件同时 更新 csr 寄存器机器模式状态寄存器 mstatus。

(2) 环境调用 – ecall : $mepc = ecall$

000000000000	000	00000	000	00000	1110011	I ecall
--------------	-----	-------	-----	-------	---------	---------

软中断，系统命令中断转而执行规定指令。

(3) 断点 – ebreak : $mepc = ebreak$

000000000001		00000	000	00000	1110011	I ebreak
--------------	--	-------	-----	-------	---------	----------

2.3 RISC-V 中断处理设计

参考 ARM 中断向量，实现非法指令异常和外部中断以及 ECALL。

- 1.外部中断 (Int) 触发中断或非法指令 (illegal) 触发异常或 ecall 系统调用
- 2.mtvec 寄存器定义的 PC 值分别 Int 为 0x0c; ecall 为 0x08; illegal 为 0x04
- 3.mepc 寄存器值更新为下一条指令的 PC 值
- 4.执行异常服务程序 (向量跳转后的程序)
- 5.执行 mret 指令，返回 mepc 保存的 PC 处继续程序流

三、实验过程和数据记录

3.1 设计实现 Datapath:

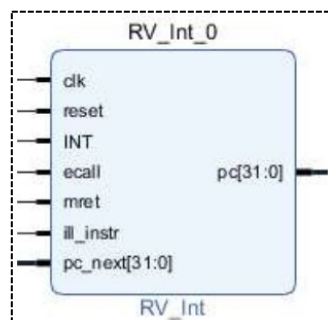
(1) Vivado 新建文件

- 利用 Vivado 新建文件，命名 OExp04-Datapath_int;

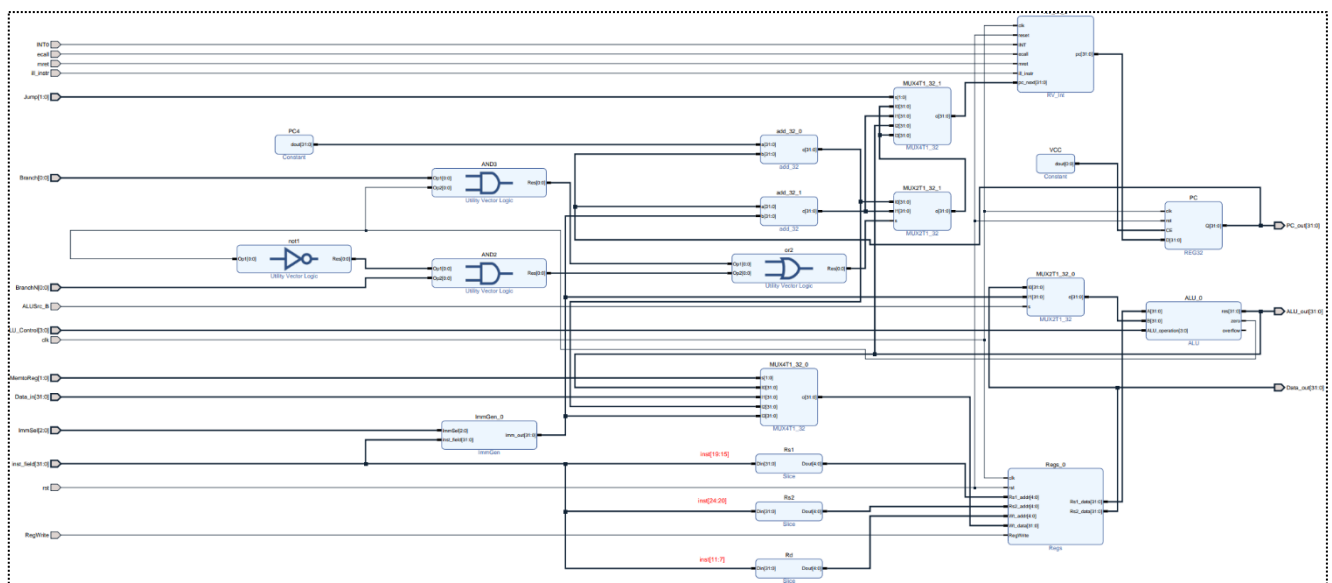
本实验仍然需要用到 Lab0 设计的 MUX 多路选择器、若干基本运算模块 (and、or、xor、nor、srl 等)，以及 Lab1 设计的 Regs。以及 Lab4-03 中修改后的 ALU、ImmGen_more 两个模块 (具体代码见实验原理)。

同时，中断的加入意味着 PC 模块需要增加：

- mtvec 寄存器型变量 (非必需)，中断和异常触发 PC 转向中断地址
 - ◆ 相当于硬件触发 Jal，用 mret 返回
- mepc 寄存器，中断和异常返回 PC 的地址
- 增加控制信号 INT (不要重复响应)、mret、ecall、ill_instr



(2) 设计 Datapath_more:



首先需要完成 RV_int 模块的设计:

```
`timescale 1ns / 1ps

module RV_int(
    input wire clk ,
    input wire reset ,
    input wire INT,
    input wire ecall ,
    input wire mret,
    input wire ill_instr,
    input wire [31:0] pc_next,
    output reg [31:0] pc,
    output reg [31:0] MEPC
);
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            MEPC <= 32'h00000000; // 复位时清除 MEPC
        end
        else if (INT | ecall | ill_instr) begin
            MEPC <= pc_next; // 中断发生时, 保存下一条指令的地址
        end
    end

    always @(*) begin
        if (reset) begin
            pc = 32'h00000000;
        end
        else if (INT) begin
            pc = 32'h0000000c; // 跳转到中断处理入口
        end
        else if (ecall) begin
            pc = 32'h00000008; // 跳转到 ecall 处理入口
        end
        else if (ill_instr) begin
            pc = 32'h00000004; // 跳转到非法指令处理入口
        end
        else if (mret) begin
            pc = MEPC; // 执行 mret 时恢复到 MEPC 保存的下一条指令
        end
        else begin // 如果没有异常, 正常更新 MEPC
            pc = pc_next;
        end
    end
end
```

逻辑原理图已给出，相较于 Datapath_more 的区别只在添加了 RV_int 模块来进行中断处理，代码实现如下：

```
`timescale 1ns / 1ps

module Datapath_int(
    input wire clk,
    input wire rst,
    input wire[31:0] inst_field,
    input wire[31:0] Data_in,
    input wire[3:0] ALU_operation,
    input wire[2:0] ImmSel,
    input wire[1:0] MemtoReg,
    input wire ALUSrc_B,
    input wire [1:0] Jump,
    input wire Branch, // beq
    input wire BranchN, // bne
    input wire RegWrite,

    input wire INT0,
    input wire ecall,
    input wire mret,
    input wire ill_instr,

    output wire[31:0] PC_out,
    output wire[31:0] Data_out,
    output wire[31:0] ALU_out,
    output wire[1023:0] all_regs_data,
    output wire[31:0] mepc
);

    wire[31:0] Imm_out;
    wire[31:0] PC_inc;
    wire[31:0] PC_imm;
    wire ALU_zero;
    wire[31:0] PC_1;
    wire[31:0] Wt_data;
    wire[31:0] PC_new;
    wire[31:0] Rs1_data;
    wire[31:0] ALU_B;
    wire[31:0] PC_int;
```



```

RV_int RV_int(
    .clk(clk),
    .reset(rst),
    .INT(INT0),
    .ecall(ecall),
    .mret(mret),
    .ill_instr(ill_instr),
    .pc_next(PC_new),
    .pc(PC_int),
    .MEPC(mepc));
ImmGen_more ImmGen(
    .ImmSel(ImmSel),
    .inst_field(inst_field),
    .Imm_out(Imm_out));
add32 add_32_0(
    .a(32'h4),
    .b(PC_out),
    .c(PC_inc));
add32 add_32_1(
    .a(PC_out),
    .b(Imm_out),
    .c(PC_imm));
MUX2T1_32 MUX2T1_32_1(
    .I0(PC_inc),
    .I1(PC_imm),
    .s((Branch & ALU_zero) | (BranchN & ~ALU_zero)),
    .o(PC_1));
MUX4T1_32 MUX4T1_32_0(
    .s(MemtoReg),
    .I0(ALU_out),
    .I1(Data_in),
    .I2(PC_inc),
    .I3(Imm_out),
    .o(Wt_data));
MUX4T1_32 MUX4T1_32_1(
    .I0(PC_1),
    .I1(PC_imm),
    .I2(ALU_out),
    .I3(PC_1),
    .s(Jump),
    .o(PC_new));
MUX2T1_32 MUX2T1_32_0(
    .I0(Data_out),
    .I1(Imm_out),

```

```

        .s(ALUSrc_B),
        .o(ALU_B));
    Regs Regs(
        .clk(clk),
        .rst(rst),
        .RegWrite(RegWrite),
        .Rs1_addr(inst_field[19:15]),
        .Rs2_addr(inst_field[24:20]),
        .Wt_addr(inst_field[11:7]),
        .Wt_data(Wt_data),
        .Rs1_data(Rs1_data),
        .Rs2_data(Data_out),
        .all_regs_data(all_regs_data));
    ALU ALU(
        .A(Rs1_data),
        .B(ALU_B),
        .ALU_operation(ALU_operation),
        .res(ALU_out),
        .zero(ALU_zero));
    REG32 PC(
        .clk(clk),
        .rst(rst),
        .CE(1'b1),
        .D(PC_int),
        .Q(PC_out));

endmodule

```

(3) Lab04-4 Datapath_int 工程结构:

- U2 : Datapath_int (Datapath_int.v) (11)
 - RV_int : RV_int (RV_int.v)
 - ImmGen : ImmGen_more (ImmGen_more.v)
 - add_32_0 : add32 (add32.v)
 - add_32_1 : add32 (add32.v)
 - MUX2T1_32_1 : MUX2T1_32 (MUX2T1_32.v)
 - MUX4T1_32_0 : MUX4T1_32 (MUX4T1_32.v)
 - MUX4T1_32_1 : MUX4T1_32 (MUX4T1_32.v)
 - MUX2T1_32_0 : MUX2T1_32 (MUX2T1_32.v)
 - Regs : Regs (Regs.v)
 - ALU : ALU (ALU.v)
 - PC : REG32 (REG32.v)

3.2 设计实现 SCPU_Ctrl_int:

(1) Vivado 新建文件

- 利用 Vivado 新建文件，命名 OExp04-SCPU_ctrl_int;

主要实现两个阶段的译码：根据指令 inst_in 对主要控制信号进行赋值分配；根据 ALU_op、Funct3/7 对 ALU 中的 ALU_Control 进行赋值。

本实验额外需要的译码是在 default 部分为 illegal 指令给予中断，并为新添加的 ecall、mret 指令（Opcode = 11100）译码。

(2) 描述设计 SCPU_ctrl_more:

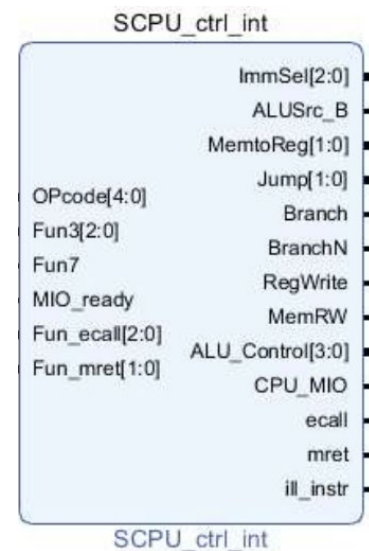
- 逻辑接口:

```
module SCPU_ctrl_int(
    input [4:0] Opcode, //Opcode
    input [2:0] Fun3, //Function
    input Fun7, //Function
    input MIO_ready, //CPU Wait
    input [2:0] Fun_ecall,
    input [1:0] Fun_mret,

    output reg [2:0] ImmSel, //立即数选择控制
    output reg ALUSrc_B, //源操作数 2 选择
    output reg [1:0] MemtoReg, //写回数据选择控制
    output reg [1:0] Jump, //jal
    output reg Branch, //beq
    output reg BranchN,
    output reg RegWrite, //寄存器写使能
    output reg MemRW, //存储器读写使能
    output reg [3:0] ALU_Control, //alu 控制
    output reg CPU_MIO,

    output reg ecall,
    output reg mret,
    output reg ill_instr
);

reg [1:0] ALUop;
wire [3:0] Fun;
assign Fun = {Fun3, Fun7};
```



- 主控制器设计:

```

always @(*) begin
    case(OPcode)
5'b01100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_1_0_0_0_00_10_000; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //ALU
5'b00000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_01_1_0_0_0_00_00_001; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //load
5'b01000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_00_0_1_0_0_00_00_010; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //store
5'b11000: begin
    case(Fun3)
3'b000: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_0_0_1_0_00_01_011; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0; end // beq
3'b001: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_00_0_0_0_1_00_01_011; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0; end // bne
default : begin ill_instr = 1'b1; ecall = 1'b0; mret = 1'b0; end
    endcase
    end
5'b11011: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_01_00_100; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //jump
5'b00100: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_00_1_0_0_0_00_11_001; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //ALU(addi;;;)
5'b11001: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b1_10_1_0_0_0_10_00_001; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //jarl
5'b01101: begin
{ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel} =
14'b0_11_1_0_0_0_00_00_000; ill_instr = 1'b0; ecall = 1'b0; mret =
1'b0;end //lui

```

```

5'b11100: begin
    {ALUSrc_B,MemtoReg,RegWrite,MemRW,Branch,BranchN,Jump,ALUop,ImmSel}
    = 14'h0;
    if(Fun_mret == 2'b00 && Fun_ecall == 3'b000) begin
        ecall = 1'b1;
        mret = 1'b0;
        ill_instr = 1'b0;
    end
    else if(Fun_mret == 2'b11 && Fun_ecall == 3'b010) begin
        mret = 1'b1;
        ecall = 1'b0;
        ill_instr = 1'b0;
    end
    else begin
        ill_instr = 1'b1;
        mret = 1'b0;
        ecall = 1'b0;
    end
    end
    default: begin ill_instr = 1'b1; ecall = 1'b0; mret = 1'b0; end
    endcase
end

```

- ALU 译码控制设计:

```

always @(*) begin
    case(ALUop)
        2'b00: ALU_Control = 4'b0010 ;
        2'b01: ALU_Control = 4'b0110 ;
        2'b10:
            case(Fun)
                4'b0000: ALU_Control = 4'b0010 ; //add
                4'b0001: ALU_Control = 4'b0110 ; //sub
                4'b1110: ALU_Control = 4'b0000 ; //and
                4'b1100: ALU_Control = 4'b0001 ; //or
                4'b0100: ALU_Control = 4'b0111 ; //slt
                4'b1010: ALU_Control = 4'b1101 ; //srl
                4'b1000: ALU_Control = 4'b1100 ; //xor
                4'b0110: ALU_Control = 4'b1001 ; //sltu
                4'b0010: ALU_Control = 4'b1110 ; //sll
                4'b1011: ALU_Control = 4'b1111 ; //sra
            default: ALU_Control = 4'bx;
            endcase
    endcase
end

```

```

2'b11:
    case(Fun3)
        3'b000: ALU_Control = 4'b0010 ; //addi
        3'b010: ALU_Control = 4'b0111 ; //slti
        3'b100: ALU_Control = 4'b1100 ; //xori
        3'b110: ALU_Control = 4'b0001 ; //ori
        3'b111: ALU_Control = 4'b0000 ; //andi
        3'b101:
            if(Fun7)
                ALU_Control = 4'b1111 ; //srai
            else
                ALU_Control = 4'b1101 ; //srli
        3'b011: ALU_Control = 4'b1001 ; //sltiu
        3'b001: ALU_Control = 4'b1110 ; //slli
        default: ALU_Control = 4'bx ;
    endcase
endcase
end

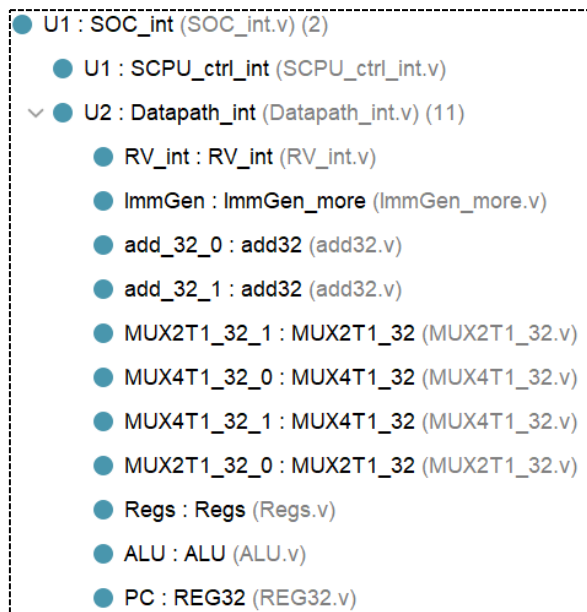
```

3.3 集成 CPU

3.3.1 搭建仿真平台

通过 3.1、3.2, 我们通过逻辑代码实现了 Datapath_int 和 SCPU_ctrl_int 模块, 对 CPU 进行了指令拓展, 我们参考 Lab4-02 的操作也能建立一个 CPU 仿真测试平台, 对相关指令进行检测。

首先对两个模块进行集成组合成 CPU:



【具体代码如下】

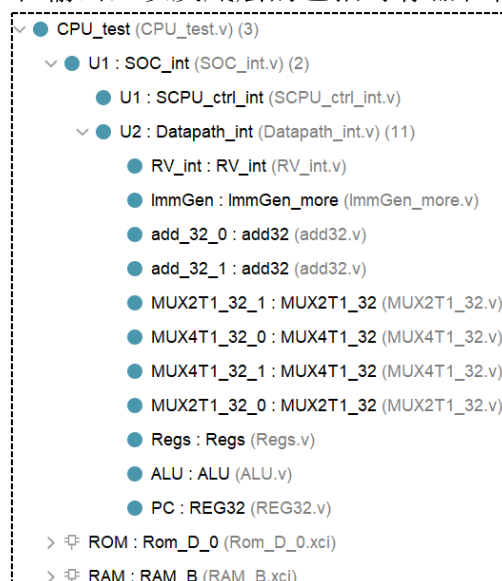
```
`timescale 1ns / 1ps
module SOC_int(
    input clk,
    input rst,
    input [31:0]Data_in,
    input [31:0]inst_in,
    input MIO_ready,
    inout INT0,
    output MemRW,
    output CPU_MIO,
    output [31:0]Data_out,
    output [31:0]PC_out,
    output [31:0]Addr_out,
    output [1023:0] all_regs_data,
    output [31:0]mepc
);
    wire [3:0]ALU_Control;
    wire [2:0]ImmSel;
    wire [1:0]MemtoReg,Jump;
    wire Branch, BranchN, RegWrite, ALUSrc_B;
    wire ecall , mret, ill_instr;
    wire[4:0] opcode;    assign opcode = inst_in[6:2];
    wire[3:0] Fun3;      assign Fun3 = inst_in[14:12];
    wire[3:0] funecall;  assign funecall = inst_in[22:20];
    wire[2:0] funmret;   assign funmret = inst_in[29:28];
    SCPU_ctrl_int U1(
        .OPcode(opcode),
        .Fun3(Fun3),
        .Fun7(inst_in[30]),
        .MIO_ready(MIO_ready),
        .Fun_ecall(funecall),
        .Fun_mret(funmret),
        .ImmSel(ImmSel),
        .ALUSrc_B(ALUSrc_B),
        .MemtoReg(MemtoReg),
        .Jump(Jump),
        .Branch(Branch),
        .BranchN(BranchN),
        .RegWrite(RegWrite),
        .MemRW(MemRW),
        .ALU_Control(ALU_Control),
        .CPU_MIO(CPU_MIO),
        .ecall(ecall),
```

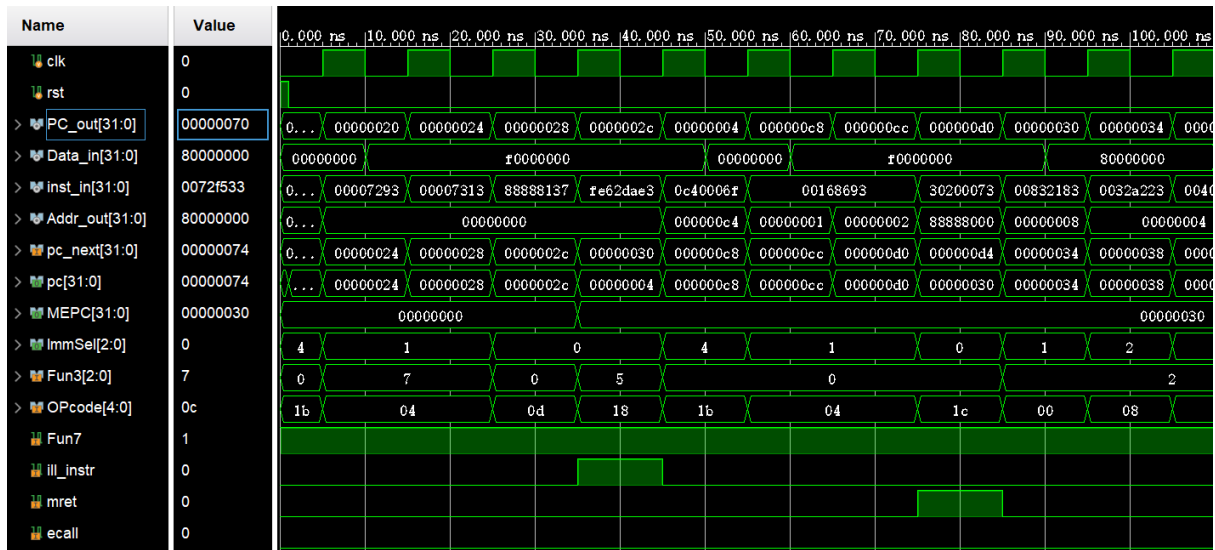
```

        .mret(mret),
        .ill_instr(ill_instr));
Datapath_int U2(
    .ALUSrc_B(ALUSrc_B),
    .ALU_operation(ALU_Control),
    .Branch(Branch),
    .BranchN(BranchN),
    .Data_in(Data_in),
    .ImmSel(ImmSel),
    .Jump(Jump),
    .MemtoReg(MemtoReg),
    .RegWrite(RegWrite),
    .clk(clk),
    .inst_field(inst_in),
    .rst(rst),
    .ALU_out(Addr_out),
    .Data_out(Data_out),
    .PC_out(PC_out),
    .all_regs_data(all_regs_data),
    .INT0(INT0),
    .ecall(ecall),
    .ill_instr(ill_instr),
    .mret(mret),
    .mepc(mepc));
endmodule

```

同样的，利用 CPU（本实验设计）、RAM、ROM 可以搭建如下图的仿真平台，初始化 RAM 的数据（D_mem.coe），初始化 ROM 的指令（I_mem_int.coe），查看 CPU 数据的输入和输出，以及底层的包括寄存器在内的各模块的状态。

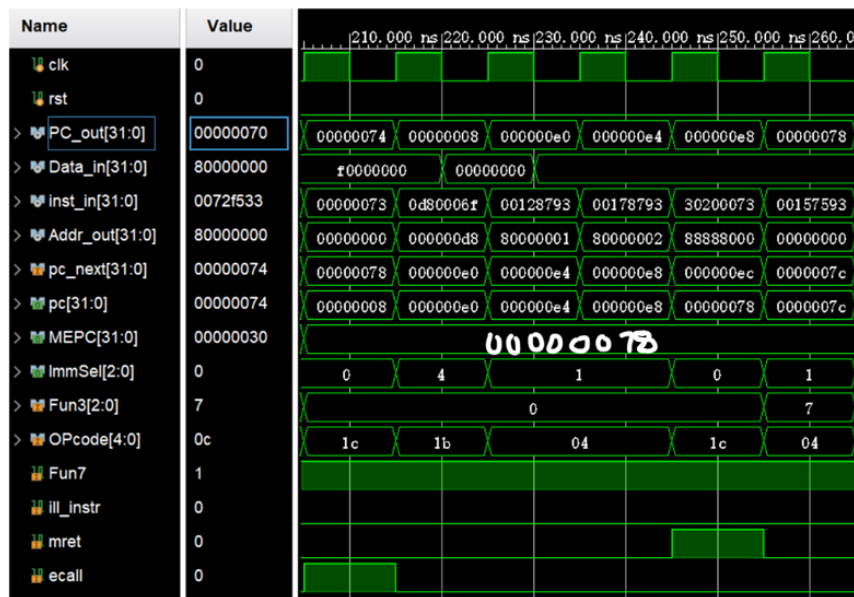




该片段中，PC = 0x2c 时指令为 0xFE62DAE3，对应机器码为 bge x5, x6, -12 不在我们的设计范围中，因此判定为 illegal instruction 需要中断。

下一个时钟周期，可以看到 PC 跳转到 ill_init 处理向量位置 0x04，同时 MEPC 记录下一指令 0x30，并转向中断处理程序 0xc8 进行处理。

处理完毕，进行 mret，返回到正常程序流。



该片段中，PC = 0x74 时指令为 0X00000073，对应机器码为 ecall，因此判定需要中断。

下一个时钟周期，可以看到 PC 跳转到 ecall 处理向量位置 0x08，同时 MEPC 记录下一指令 0x78，并转向中断处理程序 0xe0 进行处理。

处理完毕，进行 mret，返回到正常程序流。

3.3.2 DEMO 物理测试

继承替换 CPU 及子模块后，下载.bit 文件，使用 DEMO 程序进行物理验证。

本实验由于指令种类多样并且指令较为复杂，通过记录 VGA 中的数据变化，设计了测试记录表格。

- ROM 中填充指令（I_mem_int.coe）如下：

```
memory_initialization_radix=16;
memory_initialization_vector=
0200006F,0C40006F,0D80006F,0C80006F,00000033,00000033,00000033,00000033,00007293,00007313,
88888137,FE62DAE3,00832183,0032A223,00402083,01C02383,00338863,555550B7,0070A0B3,FE0098E3,
007282B3,00230333,00531463,40000033,40530433,405304B3,0080006F,00007033,0072F533,00000073,
00157593,00B51463,00006033,00A5E5B3,0015E513,00558463,00004033,00A5C633,00164613,00B61463,
00000013,0012D293,00060463,40000033,00129293,00B28463,00000013,001026B3,00503733,F5DFF06F,
00168693,00168693,30200073,40C70733,40C70733,30200073,00128793,00178793,30200073
```

- RAM 中储存的数据（D_mem.coe）如下：

```
memory_initialization_radix=16;
memory_initialization_vector=
F0000000, 000002AB, 80000000, 0000003F, 00000001, FFF70000,
0000FFFF, 80000000, 00000000, 11111111, 22222222, 33333333,
44444444, 55555555, 66666666, 77777777, 88888888, 99999999,
aaaaaaaa, bbbbbbbb, cccccccc, dddddddd, eeeeeeee, FFFFFFFF,
557EF7E0, D7BDFBD9, D7BDFBD9, DFCFFCFB, DFCFBFFF, F7F3DFFF,
FFFFDF3D, FFFF9DB9, FFFFBCFB, DFCFFCFB, DFCFBFFF, D7DB9FFF,
D7BDFBD9, D7BDFBD9, FFFF07E0, 007E0FFF, 03bdf020, 03def820,
08002300
```

部分测试 VGA 显示图片见实验结果分析

四、实验结果分析

PC	Machine Code	Code Basic	Status	Result	MEPC
0x0	0x0200006F	jal x0 32		go to 0x20	0x00
0x4	0x0C40006F	jal x0 196	ill_init	go to 0xc8	
0x8	0x0D80006F	jal x0 216	ecall_init	go to 0xe0	
0xc	0x0C80006F	jal x0 200	int_init	go to 0xd4	
0x10 - 0x1c	0x00000033	add x0 x0 x0	no use		
0x20	0x00007293	andi x5 x0 0		x5 = 0x0	
0x24	0x00007313	andi x6 x0 0		x6 = 0x0	
0x28	0x88888137	lui x2 559240		x2 = 0x88888000	
0x2c	0xFE62DAE3	bge x5 x6 -12	illegal	go to 0x04	0x30
0x30	0x00832183	lw x3 8(x6)		x3 = 0x80000000	
0x34	0x0032A223	sw x3 4(x5)		mem[1] = 0x80000000	
0x38	0x00402083	lw x1 4(x0)		x1 = 0x80000000	
0x3c	0x01C02383	lw x7 28(x0)		x7 = 0x80000000	
0x40	0x00338863	beq x7 x3 16		go to 0x50	
0x44	0x555550B7	lui x1 349525	Jump		
0x48	0x0070A0B3	slt x1 x1 x7	Jump		
0x4c	0xFE0098E3	bne x1 x0 -16	Jump		
0x50	0x007282B3	add x5 x5 x7		x5 = 0x80000000	
0x54	0x00230333	add x6 x6 x2		x6 = 0x88888000	
0x58	0x00531463	bne x6 x5 8		go to 0x60	
0x5c	0x40000033	sub x0 x0 x0	Jump		
0x60	0x40530433	sub x8 x6 x5		x8 = 0x08888000	
0x64	0x405304B3	sub x9 x6 x5		x9 = 0x08888000	
0x68	0x0080006F	jal x0 8		go to 0x70	
0x6c	0x00007033	and x0 x0 x0	Jump		
0x70	0x0072F533	and x10 x5 x7		x10 = 0x80000000	
0x74	0x00000073	ecall	ecall	go to 0x08	0x78
0x78	0x00157593	andi x11 x10 1		x11 = 0x0	
0x7c	0x00B51463	bne x10 x11 8		go to 0x84	
0x80	0x00006033	or x0 x0 x0	Jump		
0x84	0x00A5E5B3	or x11 x11 x10		x11 = 0x80000000	
0x88	0x0015E513	ori x10 x11 1		x10 = 0x80000001	
0x8c	0x00558463	beq x11 x5 8		go to 0x94	
0x90	0x00004033	xor x0 x0 x0	Jump		
0x94	0x00A5C633	xor x12 x11 x10		x12 = 0x00000001	
0x98	0x00164613	xori x12 x12 1		x12 = 0x0	
0x9c	0x00B61463	bne x12 x11 8		go to a4	
0xa0	0x00000013	addi x0 x0 0	Jump		
0xa4	0x0012D293	srl x5 x5 1		x5 = 0x40000000	
0xa8	0x00060463	beq x12 x0 8		go to 0xb0	
0xac	0x40000033	sub x0 x0 x0	Jump		
0xb0	0x00129293	slli x5 x5 1		x5 = 0x80000000	
0xb4	0x00B28463	beq x5 x11 8		go to 0xbc	
0xb8	0x00000013	addi x0 x0 0	Jump		
0xbc	0x001026B3	slt x13 x0 x1		x13 = 0x0	
0xc0	0x00503733	sltu x14 x0 x5		x14 = 0x00000001	
0xc4	0xF5DFF06F	jal x0 -164		go to 0x0	

以上表格记录了 I_mem_int.coe 中的正常程序流，并标记了中断处。

0xc8	0x00168693	addi x13 x13 1	ill_init	x13 = 0x00000001	0x30
0xcc	0x00168693	addi x13 x13 1		x13 = 0x00000002	0x30
0xd0	0x30200073	mret	mret	go to PC = MEPC	0x30
0xd4	0x40C70733	sub x14 x14 x12			
0xd8	0x40C70733	sub x14 x14 x12			
0xdc	0x30200073	mret			
0xe0	0x00128793	addi x15 x5 1	ecall	x15 = 0x80000001	0x78
0xe4	0x00178793	addi x15 x15 1		x15 = 0x80000002	0x78
0xe8	0x30200073	mret	mret	go to PC = MEPC	0x78

该表格记录了中断处理程序，其中 MEPC 保持在进入中断时程序的下一条 PC，从而能够在 mret 时重回正常程序流。

(1) Illegal instruction 中断:

```
RV32I Single Cycle CPU
pc: 0000002c inst: fe62dae3

x0: 00000000 ra: 00000000 sp: 88888000 gp: 00000000 tp: 00000000
t0: 00000000 t1: 00000000 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 00000000 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: f0000000 dmem_i_data: 00000000 dmem_addr: 00000000

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000
```

【Instruction = 0xFE62DAE3 = bge x5, x6, -12, 记录 MEPC=0x30】

```
RV32I Single Cycle CPU
pc: 00000004 inst: 0c40006f

x0: 00000000 ra: 00000000 sp: 88888000 gp: 00000000 tp: 00000000
t0: 00000000 t1: 00000000 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000000 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 000000c4 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: 00000000 dmem_i_data: 00000000 dmem_addr: 000000c4

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000
```

【Illegal 中断发生, 跳转至 0x4, 准备跳入中断处理程序】

```
RV32I Single Cycle CPU
pc: 000000d0 inst: 30200073

x0: 00000000 ra: 00000000 sp: 88888000 gp: 00000000 tp: 00000000
t0: 00000000 t1: 00000000 t2: 00000000 s0: 00000000 s1: 00000000
a0: 00000000 a1: 00000000 a2: 00000000 a3: 00000002 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 88888000 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: f0000000 dmem_i_data: 00000000 dmem_addr: 88888000

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000
```

【0xc8~0xd0 执行成功, x13=0x2, 随后跳回 0x30】

(2) ECALL 中断:

```
RV32I Single Cycle CPU
pc: 00000074 inst: 00000073
x0: 00000000 ra: 80000000 sp: 88888000 gp: 80000000 tp: 00000000
t0: 80000000 t1: 88888000 t2: 80000000 s0: 88888000 s1: 88888000
a0: 80000000 a1: 00000000 a2: 00000000 a3: 00000002 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 00000000 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: f0000000 dmem_i_data: 00000000 dmem_addr: 00000000

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000
```

【Instruction = 0x00000073 = ecall, 记录 MEPC=0x78】

```
RV32I Single Cycle CPU
pc: 00000008 inst: 0d80006f
x0: 00000000 ra: 80000000 sp: 88888000 gp: 80000000 tp: 00000000
t0: 80000000 t1: 88888000 t2: 80000000 s0: 88888000 s1: 88888000
a0: 80000000 a1: 00000000 a2: 00000000 a3: 00000002 a4: 00000000
a5: 00000000 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 000000d8 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: 00000000 dmem_i_data: 00000000 dmem_addr: 000000d8

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000
```

【Ecall 中断发生, 跳转至 0x8, 准备跳入中断处理程序】

```
RV32I Single Cycle CPU
pc: 000000e8 inst: 30200073
x0: 00000000 ra: 80000000 sp: 88888000 gp: 80000000 tp: 00000000
t0: 80000000 t1: 88888000 t2: 80000000 s0: 88888000 s1: 88888000
a0: 80000000 a1: 00000000 a2: 00000000 a3: 00000002 a4: 00000000
a5: 80000002 a6: 00000000 a7: 00000000 s2: 00000000 s3: 00000000
s4: 00000000 s5: 00000000 s6: 00000000 s7: 00000000 s8: 00000000
s9: 00000000 s10: 00000000 s11: 00000000 t3: 00000000 t4: 00000000
t5: 00000000 t6: 00000000

rs1: 00 rs1_val: 00000000
rs2: 00 rs2_val: 00000000
rd: 00 reg_i_data: 00000000 reg_wen: 0

is_imm: 0 is_auiopc: 0 is_lui: 0 imm: 00000000
a_val: 00000000 b_val: 00000000 alu_ctrl: 0 cmp_ctrl: 0
alu_res: 88888000 cmp_res: 0

is_branch: 0 is_jal: 0 is_jalr: 0
do_branch: 0 pc_branch: 00000000

mem_wen: 0 mem_ren: 0
dmem_o_data: f0000000 dmem_i_data: 00000000 dmem_addr: 88888000

csr_wen: 0 csr_ind: 000 csr_ctrl: 0 csr_r_data: 00000000
mstatus: 00000000 mcause: 00000000 mepc: 00000000 mtval: 00000000
mtvec: 00000000 mie: 00000000 mip: 00000000
```

【0xe0~0xe8 执行成功, x15=0x2, 随后跳回 0x78】

(3) INT 外部中断:

在运行过程中, 将 SW[15]调至 1, 并进行一步单步时钟, INT 外部中断发生, 跳转至 0x8, 准备跳入中断处理程序。

```
RV32I Single Cycle CPU
pc: 0000000c   inst: 0c80006f
x0: 00000000   ra: 80000000   sp: 88888000   gp: 80000000   tp: 00000000
t0: 00000000   t1: 88888000   t2: 80000000   s0: 08888000   s1: 08888000
a0: 00000000   a1: 80000000   a2: 00000000   a3: 00000002   a4: 00000000
s4: 00000000   s5: 00000000   s6: 00000000   s7: 00000000   s8: 00000000
s9: 00000000   s10: 00000000   s11: 00000000   t3: 00000000   t4: 00000000
t5: 00000000   t6: 00000000

rs1: 00   rs1_val: 00000000
rs2: 00   rs2_val: 00000000
rd: 00   reg_i_data: 00000000   reg_wen: 0

is_imm: 0   is_auiopc: 0   is_lui: 0   imm: 00000000
a_val: 00000000   b_val: 00000000   alu_ctrl: 0   cmp_ctrl: 0
alu_res: 00000008   cmp_res: 0

is_branch: 0   is_jal: 0   is_jalr: 0
do_branch: 0   pc_branch: 00000000

mem_wen: 0   mem_ren: 0
dmem_o_data: 00000000   dmem_i_data: 08888000   dmem_addr: 00000008

csr_wen: 0   csr_ind: 000   csr_ctrl: 0   csr_r_data: 00000000
mstatus: 00000000   mcause: 00000000   mepc: 00000000   mtval: 00000000
mtvec: 00000000   mie: 00000000   mip: 00000000
```

随后 0xd4~0xdc 执行成功, 随后跳回原程序流。

```
RV32I Single Cycle CPU
pc: 000000d8   inst: 40c70733
x0: 00000000   ra: 80000000   sp: 88888000   gp: 80000000   tp: 00000000
t0: 00000000   t1: 00000000   t2: 80000000   s0: 08888000   s1: 08888000
a0: 00000001   a1: 80000000   a2: 00000000   a3: 00000000   a4: 00000001
s4: 00000000   s5: 00000000   s6: 00000000   s7: 00000000   s8: 00000000
s9: 00000000   s10: 00000000   s11: 00000000   t3: 00000000   t4: 00000000
t5: 00000000   t6: 00000000

rs1: 00   rs1_val: 00000000
rs2: 00   rs2_val: 00000000
rd: 00   reg_i_data: 00000000   reg_wen: 0

is_imm: 0   is_auiopc: 0   is_lui: 0   imm: 00000000
a_val: 00000000   b_val: 00000000   alu_ctrl: 0   cmp_ctrl: 0
alu_res: 00000001   cmp_res: 0

is_branch: 0   is_jal: 0   is_jalr: 0
do_branch: 0   pc_branch: 00000000

mem_wen: 0   mem_ren: 0
dmem_o_data: f0000000   dmem_i_data: 00000000   dmem_addr: 00000001

csr_wen: 0   csr_ind: 000   csr_ctrl: 0   csr_r_data: 00000000
mstatus: 00000000   mcause: 00000000   mepc: 00000000   mtval: 00000000
mtvec: 00000000   mie: 00000000   mip: 00000000
```

(4) 最终寄存器堆数值:

```
RV32I Single Cycle CPU
pc: 000000c4   inst: f5dff06f
x0: 00000000   ra: 80000000   sp: 88888000   gp: 80000000   tp: 00000000
t0: 80000000   t1: 88888000   t2: 80000000   s0: 08888000   s1: 08888000
a0: 80000001   a1: 80000000   a2: 00000000   a3: 00000000   a4: 00000001
s4: 00000000   s5: 00000000   s6: 00000000   s7: 00000000   s8: 00000000
s9: 00000000   s10: 00000000   s11: 00000000   t3: 00000000   t4: 00000000
t5: 00000000   t6: 00000000

rs1: 00   rs1_val: 00000000
rs2: 00   rs2_val: 00000000
rd: 00   reg_i_data: 00000000   reg_wen: 0

is_imm: 0   is_auiopc: 0   is_lui: 0   imm: 00000000
a_val: 00000000   b_val: 00000000   alu_ctrl: 0   cmp_ctrl: 0
alu_res: ffffffff   cmp_res: 0

is_branch: 0   is_jal: 0   is_jalr: 0
do_branch: 0   pc_branch: 00000000

mem_wen: 0   mem_ren: 0
dmem_o_data: f0000000   dmem_i_data: 00000000   dmem_addr: ffffffff

csr_wen: 0   csr_ind: 000   csr_ctrl: 0   csr_r_data: 00000000
mstatus: 00000000   mcause: 00000000   mepc: 00000000   mtval: 00000000
mtvec: 00000000   mie: 00000000   mip: 00000000
```

最后呈上无 INT 外部中断, 正确跑完 I_mem_int.coe 的结果。

五、讨论与心得

思考题 1：设计 SCPU_ctrl_int 时，所增加的特权指令与 RV32I 基本指令译码有何区别？

新增加的 特权指令，不依赖于基本指令中所用的控制信号 ALUSrc_B, MemtoReg, RegWrite, MemRW, Branch, BranchN, Jump, ALUop, ImmSel 等。

而主要特权指令决定于 32 位指令中的 Fun_mret 和 Fun_ecall，以及未在基本指令译码范围出现导致 default 中 ill_instr 的变化。

所以说目前新增加的 特权指令只需要 3 个控制信号就能正确执行，相对来说译码更加简洁明了（虽然可能是加的比较少）。

思考题 2：指令集规定的中断寄存器 mepc、mtvec 均为时钟控制寄存器，本实验在设计 RV_int 时若也都设计为寄存器，能实现 PC 的正常跳转吗？

不能正常跳转，如果 RV_int 也设计成寄存器，可以考虑一种情况，在同一个时钟上升沿和下降沿之间发生一次中断，那么在下一次时钟上升沿前 RV_int 中的数值就不能改变，也就无法探测到异常的发生。

这就达不到中断处理的要求，PC 跳转不正常。

单周期 CPU 中断实验结束（太折磨了。。），那接下来就要做流水线 CPU 了。在做单周期的一系列实验中，主要时间和工作集中在 Datapath、SCPU_ctrl 的调整、debug 上，做完以后基本上对每个指令的通路以及控制信号都比较熟悉了。

希望后续流水线 CPU 能够友善一点。

【中间存在一个未解决的问题，如果 ROM 中存在 sw 指令，很大概率会导致内存出错，导致后续 lw 数据出现问题？但是多烧几次以后也有概率获得正确结果，真的太折磨了。】